

CURSO REACT Y NODE.JS - CLASE 1 DE 6

Preparación del entorno y primer Hola Mundo

Guía para alumnado principiante: instalación de herramientas, configuración inicial y creación del primer proyecto React con Vite.

Duración

5 horas

Nivel

Inicio absoluto

Objetivo

De cero a una app React visible

Índice

1. Objetivos de la clase
2. Mapa mental del stack
3. Herramientas necesarias
4. Git, GitHub y VS Code
5. Instalación de Node.js
6. Terminal básica
7. Crear el primer proyecto
8. Hola Mundo en React
9. Errores habituales
10. Ejercicios

1. Objetivos de la clase

Esta primera clase no busca crear una aplicación compleja. Busca que todas las personas tengan el mismo entorno preparado y entiendan qué papel cumple cada herramienta.

AL TERMINAR

- El alumnado tendrá instalado Visual Studio Code.
- El alumnado tendrá instalado Git y una cuenta de GitHub.
- El alumnado habrá configurado su nombre y correo de Git.
- El alumnado tendrá instalado Node.js y npm.
- El alumnado sabrá abrir una terminal en la carpeta correcta.
- El alumnado habrá creado un primer proyecto React con Vite.
- El alumnado habrá modificado el componente principal para mostrar un Hola Mundo.

Es importante ir despacio. En desarrollo web moderno no se empieza escribiendo solo un archivo HTML. Antes necesitamos un entorno de trabajo que nos permita usar módulos, instalar librerías, ejecutar servidores de desarrollo y preparar la aplicación para producción.

2. Mapa mental del stack

Antes de instalar nada conviene entender las piezas. El stack que usaremos durante el curso tiene dos partes:

Parte	Qué hace	Herramientas
Frontend	Es la interfaz que ve la persona usuaria en el navegador.	React, Vite, JavaScript, HTML, CSS

Parte	Qué hace	Herramientas
Backend	Es el servidor que recibirá peticiones, gestionará datos y devolverá respuestas.	Node.js, Express, JSON, API REST

IDEA CLAVE

React no entra directamente en los archivos del backend. React se comunicará con el backend haciendo peticiones HTTP. Más adelante veremos llamadas como `fetch('/api/contacto')`.

Cómo queda la estructura del curso

```
00_PROYECTO_PROFE/  
  public/      archivos públicos del frontend  
  src/         código React  
  package.json dependencias y scripts del frontend  
  vite.config.js configuración de Vite  
  Backend/     backend Node/Express, que se hará más adelante
```

La raíz del proyecto será el frontend. La carpeta `Backend/` queda preparada para la parte del servidor.

3. Herramientas necesarias

3.1 Visual Studio Code

Visual Studio Code será el editor principal del curso. No es React, no es Node y no ejecuta la aplicación por sí solo. Es la herramienta donde escribimos y organizamos el código.

1. Entrar en <https://code.visualstudio.com/>.
2. Descargar la versión para Windows.
3. Instalar aceptando las opciones por defecto.
4. Marcar, si aparece, la opción de abrir carpetas con VS Code desde el explorador.

3.2 Extensiones recomendadas para VS Code

Las extensiones añaden ayudas al editor. No conviene instalar demasiadas al principio. Para esta formación bastan estas:

Extensión	Para qué sirve
Spanish Language Pack	Pone VS Code en castellano si el alumnado lo prefiere.
ESLint	Ayuda a detectar errores o malas prácticas en JavaScript y React.
Prettier	Formatea el código para que quede ordenado y consistente.
GitHub Pull Requests and Issues	Facilita la integración de GitHub dentro de VS Code.
Material Icon Theme	Opcional. Mejora la lectura visual de carpetas y archivos.

CRITERIO DIDÁCTICO

En una primera clase no necesitamos convertir VS Code en un entorno lleno de extensiones. El objetivo es entender el flujo básico: carpeta, terminal, comandos, archivos y navegador.

3.3 Git

Git es la herramienta que nos permite guardar el historial del proyecto. GitHub es una plataforma online donde podemos subir repositorios. No son lo mismo.

1. Entrar en <https://git-scm.com/download/win>.
2. Descargar e instalar Git para Windows.

3. Durante la instalación, dejar las opciones por defecto salvo indicación del profesor.
4. Cerrar y abrir de nuevo la terminal para que el comando `git` esté disponible.

Comprobación:

```
git --version
```

4. Git, GitHub y VS Code

4.1 Crear cuenta de GitHub

1. Entrar en <https://github.com/>.
2. Crear una cuenta con un correo que se vaya a conservar.
3. Verificar el correo si GitHub lo solicita.
4. Guardar usuario y contraseña en un gestor seguro.

4.2 Configurar Git desde consola

Antes de subir código, Git necesita saber quién hace los cambios. Esta configuración se hace una vez por ordenador.

```
git config --global user.name "Nombre Apellido"  
git config --global user.email "correo-usado-en-github@example.com"  
git config --global init.defaultBranch main
```

Para comprobar la configuración:

```
git config --global --list
```

IMPORTANTE

El correo configurado en Git debería coincidir con un correo asociado a la cuenta de GitHub. Así los commits aparecerán correctamente vinculados al perfil.

4.3 Conectar GitHub con VS Code

Hay dos caminos habituales. Para un curso introductorio podemos usar el más simple:

Opción A - Desde VS Code

1. Abrir VS Code.
2. Ir al icono de cuentas, abajo a la izquierda.

3. Elegir iniciar sesión con GitHub.
4. Autorizar VS Code en el navegador.

Opción B - Desde consola con GitHub CLI

GitHub CLI es una herramienta opcional para trabajar con GitHub desde la terminal. Si está instalada, se puede comprobar con:

```
gh --version
```

Si no está instalada, se puede instalar desde <https://cli.github.com/>.

Después, iniciar sesión:

```
gh auth login  
gh auth status
```

NO CONFUNDIR

`git config` configura el nombre y correo de los commits. `gh auth login` inicia sesión en GitHub desde la terminal. Son pasos relacionados, pero no son lo mismo.

5. Instalación de Node.js

Node.js permite ejecutar JavaScript fuera del navegador. En este curso lo necesitamos por dos motivos:

- para usar herramientas de desarrollo como Vite;
- para crear más adelante el backend con Node y Express.

5.1 Instalación

1. Entrar en <https://nodejs.org/>.
2. Descargar la versión LTS recomendada.
3. Instalar dejando las opciones por defecto.
4. Cerrar y abrir de nuevo la terminal.

5.2 Comprobar Node y npm

npm se instala junto con Node. npm sirve para instalar paquetes y ejecutar scripts del proyecto.

```
node -v  
npm -v
```

Si ambos comandos devuelven una versión, el entorno está preparado.

Comando	Qué comprueba
<code>node -v</code>	Que Node.js está instalado.
<code>npm -v</code>	Que npm está instalado.

6. Terminal básica

La terminal es una forma de comunicarnos con el ordenador mediante comandos. Al principio puede parecer menos cómoda que hacer clic, pero en desarrollo web es imprescindible.

Comando PowerShell	Qué hace
<code>Get-Location</code>	Muestra en qué carpeta estamos.
<code>Get-ChildItem</code>	Lista archivos y carpetas.
<code>cd nombre-carpeta</code>	Entra en una carpeta.

Comando PowerShell	Qué hace
<code>cd ..</code>	Sube un nivel.
<code>mkdir Nombre</code>	Crea una carpeta.
<code>code .</code>	Abre la carpeta actual en VS Code.

MINI PRÁCTICA

1. Crear una carpeta llamada `curso-react-node`.
2. Entrar dentro con `cd curso-react-node`.
3. Comprobar la ubicación con `Get-Location`.
4. Abrir la carpeta con `code .`

7. Crear el primer proyecto React con Vite

Vite es la herramienta que prepara el proyecto. React será la librería con la que construiremos la interfaz.

7.1 Crear el proyecto en la carpeta actual

Desde la carpeta vacía del curso ejecutamos:

```
npm create vite@latest . -- --template react
```

Parte	Significado
<code>npm</code>	Ejecuta herramientas del ecosistema Node.
<code>create vite@latest</code>	Usa el creador oficial de proyectos Vite.
<code>.</code>	Indica que queremos crear el proyecto en la carpeta actual.
<code>--template react</code>	Indica que queremos una plantilla de React.

7.2 Instalar dependencias

Vite crea archivos, pero todavía hay que instalar las librerías.

```
npm install
```

Este comando lee `package.json` y descarga las dependencias dentro de `node_modules`.

7.3 Arrancar el servidor de desarrollo

```
npm run dev
```

Vite mostrará una URL parecida a:

```
http://localhost:5173/
```

Esa dirección solo funciona mientras el servidor de desarrollo está arrancado.

CÓMO PARAR EL SERVIDOR

En la terminal donde se está ejecutando Vite, pulsar `Ctrl + C`. Si pregunta si queremos terminar el proceso, responder que sí.

8. Hola Mundo en React

8.1 Archivos que debemos mirar primero

Archivo	Función
<code>index.html</code>	HTML base que carga la aplicación.
<code>src/main.jsx</code>	Punto de entrada de React.
<code>src/App.jsx</code>	Primer componente principal.
<code>src/index.css</code>	Estilos globales.

8.2 Qué es un componente

Para empezar, podemos pensar que un componente es una función que devuelve una parte de la interfaz.

```
function App() {  
  return (  
    <main>  
      <h1>Hola Mundo</h1>  
      <p>Mi primera aplicación con React y Vite.</p>  
    </main>  
  )  
}  
  
export default App
```

8.3 JSX

JSX se parece a HTML, pero vive dentro de JavaScript. Por eso podemos mezclar estructura visual con datos.

```
const nombreCurso = 'React y Node.js'  
  
function App() {  
  return <h1>Hola {nombreCurso}</h1>  
}
```

Las llaves `{ }` permiten insertar valores JavaScript dentro del JSX.

8.4 Imports y exports

En proyectos modernos no trabajamos con un único archivo gigante. Dividimos el código en archivos y usamos imports y exports.

```
import App from './App.jsx'
```

Ese import significa: "trae el componente App desde el archivo App.jsx".

```
export default App
```

Ese export significa: "permite que otros archivos importen este componente".

9. Errores habituales

Problema	Posible causa	Solución
<code>node</code> no se reconoce	Node no está instalado o la terminal se abrió antes de instalarlo.	Instalar Node, cerrar la terminal y abrir una nueva.
<code>npm</code> no se reconoce	Node no está instalado correctamente.	Reinstalar Node LTS y comprobar con <code>node -v</code> .
No se abre <code>localhost:5173</code>	El servidor Vite no está arrancado.	Ejecutar <code>npm run dev</code> .
Estoy en la carpeta equivocada	La terminal no está ubicada en la raíz del proyecto.	Usar <code>Get-Location</code> y <code>cd</code> .
La pantalla no cambia	Se editó otro archivo o no se guardó.	Guardar el archivo y comprobar que se está editando <code>src/App.jsx</code> .

10. Ejercicios de clase

EJERCICIO 1

Comprobar el entorno

Ejecuta estos comandos y anota el resultado:

```
node -v
npm -v
git --version
```

Objetivo: confirmar que las herramientas básicas están instaladas.

EJERCICIO 2

Configurar Git

Configura tu nombre y correo:

```
git config --global user.name "Tu nombre"
git config --global user.email "tu-correo@example.com"
```

Comprueba el resultado:

```
git config --global --list
```

EJERCICIO 3

Crear el proyecto Vite

1. Crea una carpeta vacía para el curso.
2. Abre esa carpeta en VS Code.
3. Abre la terminal integrada.
4. Ejecuta `npm create vite@latest . -- --template react`.
5. Ejecuta `npm install`.
6. Ejecuta `npm run dev`.
7. Abre la URL que indique Vite.

EJERCICIO 4

Modificar el Hola Mundo

Cambia el contenido de `src/App.jsx` para que muestre:

- tu nombre;
- el nombre del curso;
- una frase sobre lo que esperas aprender.

Ejemplo orientativo:

```
const nombre = 'Ana'

function App() {
  return (
    <main>
      <h1>Hola, soy {nombre}</h1>
      <p>Estoy aprendiendo React y Node.js.</p>
    </main>
  )
}
```

```
}  
  
export default App
```

EJERCICIO 5

Explicar con tus palabras

Responde brevemente:

1. ¿Para qué sirve Node.js en este curso?
2. ¿Para qué sirve npm?
3. ¿Qué hace Vite?
4. ¿Qué archivo monta React en el navegador?
5. ¿Qué significa escribir JavaScript dentro de `{ }` en JSX?

Fin de la clase 1. La siguiente clase añade una estructura básica con rutas internas, página de contacto y carpeta Backend preparada.